Modernizing a 2M+ LOC Java Monolith with Agentic AI

What Worked, What Failed, and How We Recovered

Marek Dominiak



Marek Dominiak

I run training in EventStorming, Architecture, DDD, and TDD

19 years of professional experience

Co-owner and instructor at trainitek

Hands-on Architect and Team Lead at MINTRA

https://www.linkedin.com/in/marekdominiak

Agenda

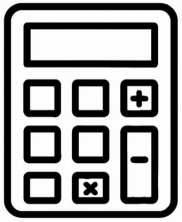
- The modernization problem
- Agentic AI in practice
- V2 meets production
- Building the harness
- V3: simpler, safer, in production
- Lessons Learned
- Q/A



The modernization problem

The system we had to modernize

- The system developed over the last 17 years
- Team size ~25-30 people during this time
- ~13 different services
- Number of files: ~64K
- Lines of code: > 2M
- Number of user accounts: ~ 3M



Why did we need to upgrade?

- Worsened Developer Experience



- Increased cost of keeping things secure

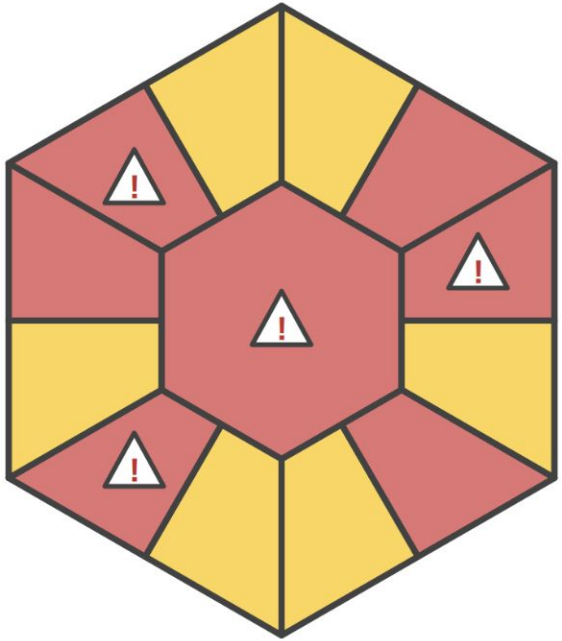


- Missed benefits from JVM upgrades



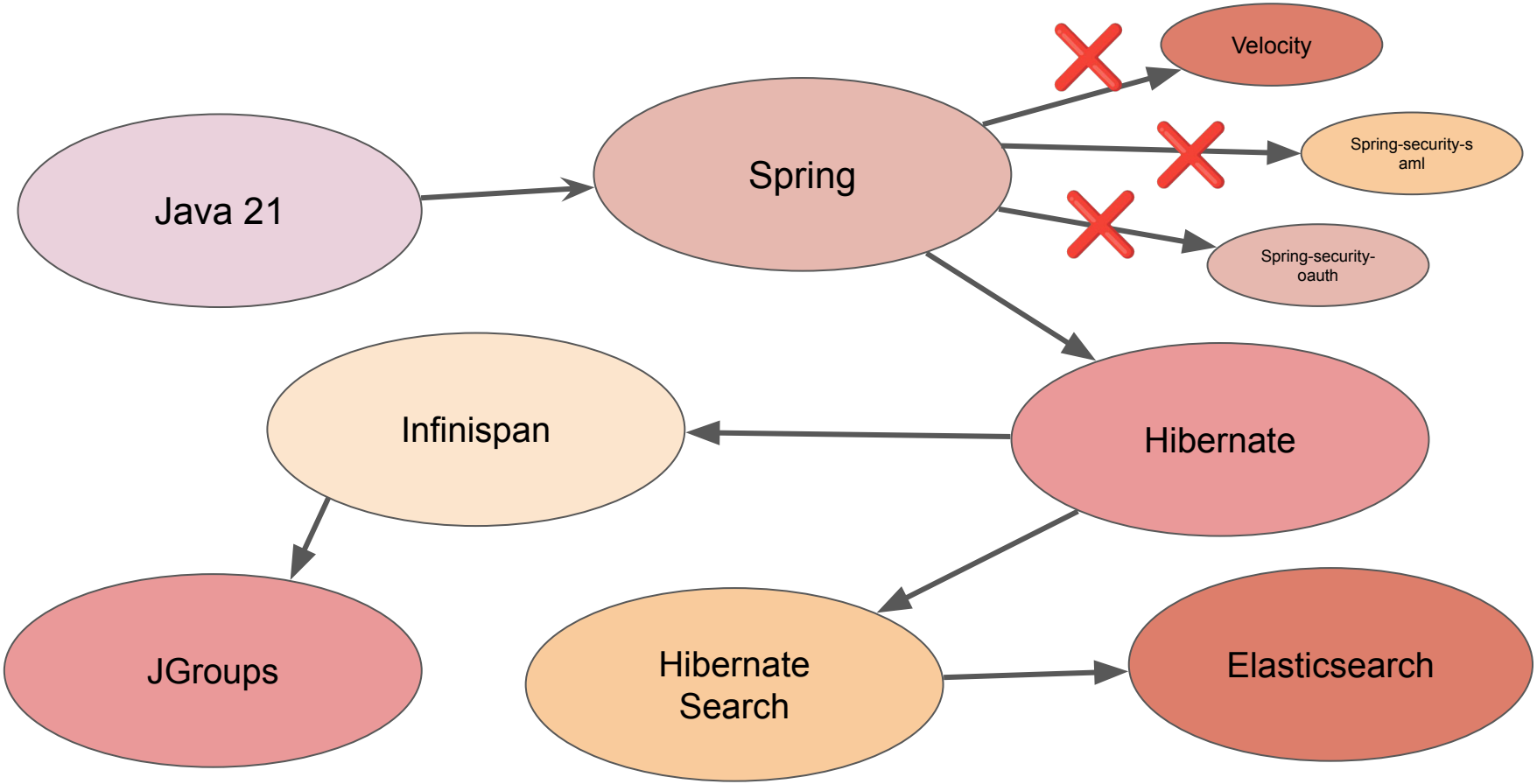
- Increased costs related to running outdated versions on Cloud (e.g. Mysql on AWS)



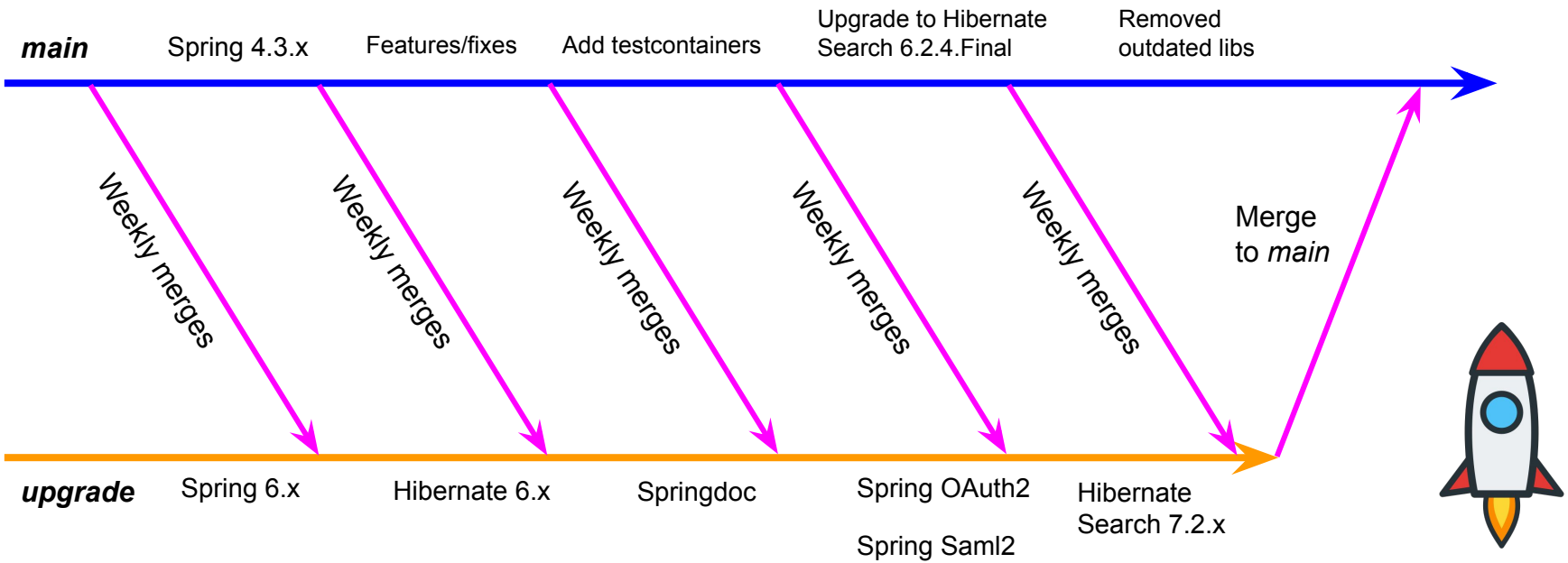


Before: one monolith, many parts, many of them with issues.

Chained Dependencies

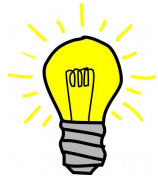


Our strategy: an upgrade branch with weekly merges



What the migration taught us

- Upgrades come in chains: one library pulls the next
- Some libraries are simply gone. Replace them or, if they’re small, “vibe code” your own (last resort)
- Old code hides old business rules
- Tools help, but they don’t know how your production behaves
- At some point the question changes: upgrade it, or finally replace it?



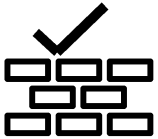
Agentic AI in practice

Our starting point

What we had

- **Tests we trust:** unit and integration, Testcontainers
- **CI** on every merge request
- **ADRs** for important decisions
- **Monitoring** in production
- **A team** that knows the domain and the code

What we didn’t have



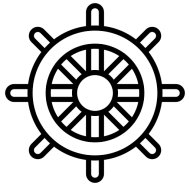
- Enough people for the modernization
- Time, next to new features and integrations
- A clear idea of how long it would take

Agents made us faster because these were already in place.

Meanwhile, on master

- **Upgrades**: Elasticsearch 7.10, MySQL 8.4, AWS SDK v2, a shrinking CVE backlog
- **Less code**: over 150k lines of dead code, modules and libraries removed
- **Decision made**: authentication moves to a separate service
- **Kubernetes-ready**: Docker image, Helm chart and EKS deployment (moving from Elastic Beanstalk)
- Older components rewritten, one of them **the indexing queues**





My merged MRs, 4 weeks of work

Merged !2753

Remove dead code and unused test fixtures

106 files +85 −1,575,684 *mostly CSV test fixtures*

Merged !2754 → feature/tp-stack-upgrade

Port master to Java 21 / Spring 6 / Hibernate 6

1,553 files +64,427 −34,456 *166 conflicts*

Merged !2678

Remove obsolete setup scripts and config profiles

193 files +184 −19,126

Merged !2626

Empty local database from the production schema

9 files +18,969 −3 *mostly generated schema*

Merged !2748

Remove tests that give no value

20 files +3 −3,126

Merged !2696

Replace S3Mock with the Floci S3 emulator

15 files +259 −1,555

Merged !2737

Group JMS queues for several worker nodes

31 files +465 −139

Merged !2736

Faster test MySQL and Elasticsearch

21 files +200 −48 *CI: 1 h 55 → 1 h 28*

22 MRs in total. Done in a fraction of my time. Without agents, not possible.

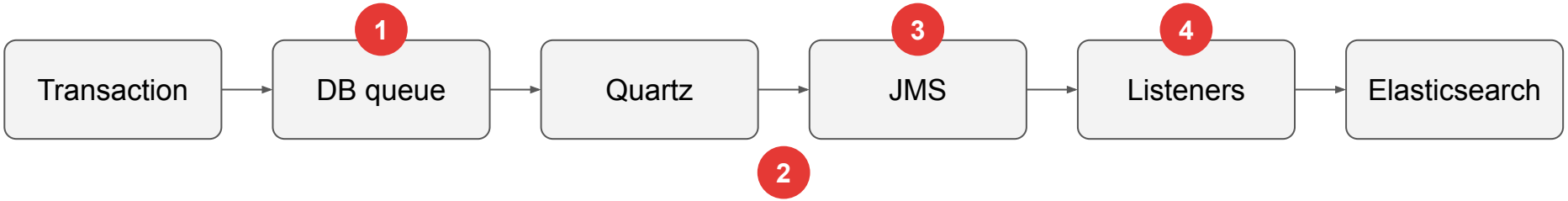
The indexer we wrote 14 years ago

- Over time, our indexing stopped being good enough
- We kept it alive for years
- We patched it whenever it broke
- But at our scale, it started to bother everyone:
  - Some customers were unhappy about search
  - Support was on the edge
  - Developers were not happy





How V1 worked



1. **Duplicates:** the same entity is queued again and again, so queues get very long
2. **Double asynchronicity:** Quartz batches, then JMS delivers: two async hops
3. **Priorities lost:** JMS ignores them, so urgent changes wait behind old background work
4. **Scaling:** adding consumers needed a server restart

Too many moving parts, and we lost control of priorities.

Architecture Decision Record (ADR)

- We wrote an ADR for the rewrite of the indexing queues
- The Architecture Board approved it
- Goal: simpler, full control over priorities, scaling without restarts

TP-013 - Improve control of priorities, and performance of Indexing Queues

By Marek Dominiak · 6 min · Listen · 18 · Add a reaction

Status	APPROVED IMPLEMENTED
Impact	MEDIUM
Drivers	Maintainability, Flexibility, Fault Tolerance
Approver	Architecture board:
Due date	Production: Jul 29, 2026

What the ADR decided

Rejected

Option 1

Hibernate Search outbox polling

+ Out of the box, battle-tested
+ Deduplication per transaction
− Still incubating
− We lose priorities and fairness

Chosen

Option 2

MySQL queue, no JMS

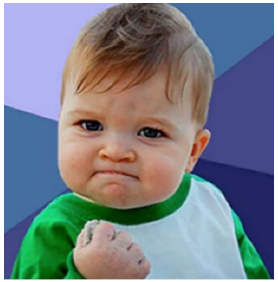
+ We stay in control of priorities and fairness
+ SELECT … FOR UPDATE SKIP LOCKED: many consumers, no contention
− We build polling and deduplication ourselves

Remove JMS. Let the database do the queueing.

V2 implementation

- After trying different AI tools and seeing good results with Opus 4.5, we finally got Claude Code
- I had just learned about Ralph loops
- I started reimplementing the indexer
- Initial results were really good
- I was honestly quite happy





My Ralph loops

Command

```
export CLAUDE_CODE_EFFORT_LEVEL=max;
for i in {1..10}; do cat architecture/v2/prompt.md
| wclaude --model claude-opus-4-6 -p; done
```

prompt.md

Study `trainingportal/architecture/v2/plan.md` and `trainingportal/architecture/v2/progress.md`, then perform only the next task to be done. The task can be completed only if the relevant tests are green.

After the task:

- update `progress.md`
- commit only the relevant changes when explicitly asked to commit
- report the step number, current time, model name, test runtime, and context usage

Ref: https://ghuntley.com/loop/

My Ralph loops

Three files:

prompt.md - the prompt from the previous slide

plan.md - plan made with */plan* and with person review based on ADR

progress.md - the state tracker of the implementation



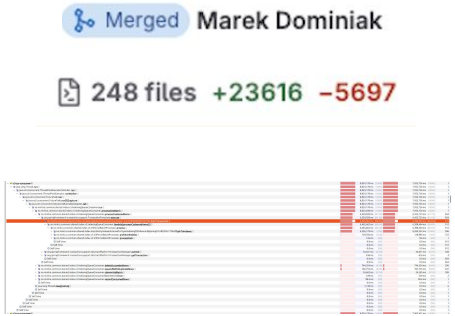
Step 50 — Preserve error state for permanently failed items on re-enqueue. 184 tests pass.	Marek Dominiak	Yesterday 12:59 PM
Step 49 — Error Rate Alarm. 182 tests pass.	Marek Dominiak	Yesterday 12:45 PM
Step 48 — Populate/Re-index All Items. 174 tests pass.	Marek Dominiak	Yesterday 12:21 PM
Step 47 — Replace wildcard imports in test files. 174 tests pass.	Marek Dominiak	Yesterday 7:07 AM
Step 46 — Replace wildcard imports with explicit imports. 174 tests pass.	Marek Dominiak	Yesterday 6:53 AM
Step 45 — Update docs and Postman for Steps 41-44. 174 tests pass.	Marek Dominiak	Yesterday 6:32 AM
Step 44 — Production-ready auto-scale defaults. 174 tests pass.	Marek Dominiak	Yesterday 6:25 AM
Step 43 — Remove expensive stale item COUNT queries. 174 tests pass.	Marek Dominiak	Yesterday 5:54 AM
Step 42 — Wall-clock batch time stats. 182 tests pass.	Marek Dominiak	Yesterday 5:40 AM
Step 41 — In-memory staleness check (zero DB queries). 182 tests pass. Co-Authored-By: Claude Opus 4.6 <noreply@anthropic.com>	Marek Dominiak	3/10/26, 5:39 PM
Step 40 — Update docs and Postman collection. 182 tests pass. Co-Authored-By: Claude Opus 4.6 <noreply@anthropic.com>	Marek Dominiak	3/10/26, 1:48 PM
Step 39 DRY review. 182 tests pass. Co-Authored-By: Claude Opus 4.6 <noreply@anthropic.com>	Marek Dominiak	3/10/26, 1:33 PM
Step 37 — Consolidate contract-expand scripts. 211 tests pass. Co-Authored-By: Claude Opus 4.6 <noreply@anthropic.com>	Marek Dominiak	3/10/26, 12:39 PM
Step 36 — Improve observability: commit/flush timing + staleness check throttling. (1) Commit/flush timing: Stopwatch wrapping transactionTemplate.execute() captures Hiber	Marek Dominiak	3/10/26, 12:15 PM
Step 35 — Remove autoTuneBatchSize feature (keep autoScaling of consumers). Removed batch size auto-tuning: fields/methods from IndexerProperties, IndexConsumer, Cor	Marek Dominiak	3/10/26, 11:44 AM
Step 34 — Include IndexConfiguration (batchSize) in properties REST endpoints. Co-Authored-By: Claude Opus 4.6 <noreply@anthropic.com>	Marek Dominiak	3/10/26, 11:36 AM
Step 33 — Add comprehensive Javadoc to IndexerProperties and configuration classes. Co-Authored-By: Claude Opus 4.6 <noreply@anthropic.com>	Marek Dominiak	3/10/26, 11:28 AM
Step 32 - `EntityIndexQueueRepositoryImpl.java` — Replaced `fetchRemainingExcludingOverrides` (NOT IN query) with `fetchByEntityClassCount` (per-class queries mirroring	Marek Dominiak	3/9/26, 3:12 PM
Step 31 — Soft-delete-aware fetch performance indexes. Add processedAt as leading column to all composite fetch indexes so MySQL can skip soft-deleted rows via index se	Marek Dominiak	3/9/26, 2:05 PM
Step 30 - Eliminate per-entity-class fetch loop. Replace 25+ SELECT FOR UPDATE SKIP LOCKED queries per batch with single NOT IN query. Co-Authored-By: Claude Opus 4.6	Marek Dominiak	3/9/26, 1:53 PM
Step 29 — Entity fetch optimization + status config	Marek Dominiak	3/9/26, 12:23 PM
Step 28 — Documentation & Postman audit	Marek Dominiak	3/6/26, 1:41 PM
Step 27 — Developer FAQ: Group stats lifecycle	Marek Dominiak	3/6/26, 1:22 PM
Step 26 — REST stop-all / start-all endpoints	Marek Dominiak	3/6/26, 1:09 PM
Step 25 — Periodic stats reconciliation	Marek Dominiak	3/6/26, 12:49 PM
Step 24 — Default max consumers 10 → 3	Marek Dominiak	3/6/26, 12:20 PM
Step 23 — Plan for bullet-proofing indexing infrastructure	Marek Dominiak	3/6/26, 12:11 PM
Step 22 — Troubleshooting guide + solution comparison docs sync	Marek Dominiak	3/6/26, 6:51 AM
Step 21 — Reconcile group stats on coordinator startup	Marek Dominiak	3/6/26, 6:42 AM
Step 20 — ADR + Developer FAQ update after in-memory cache	Marek Dominiak	3/6/26, 6:24 AM
Step 19 — Tests for in-memory pendingDecrements cache	Marek Dominiak	3/6/26, 6:12 AM
Step 18 — Wire flushPendingDecrements into indexCoordinator lifecycle	Marek Dominiak	3/6/26, 6:05 AM
Step 17 — In-memory pendingDecrements cache for decrementGroupStats	Marek Dominiak	3/6/26, 5:53 AM
Step 16 — Plan: In-memory group stats cache to eliminate decrementGroupStats bottleneck	Marek Dominiak	3/6/26, 5:25 AM

Keep the sessions small

V2 meets production

Around 250 Ralph loops later…

- Test coverage close to 100% + performance tests
- I tested heavily
- I had VisualVM profiling sessions
- I was happy to show it to the team
- We put it on beta
- I got feedback and fixed a few things
- We went to **staging**
- And shortly after **production**





Production challenged my confidence

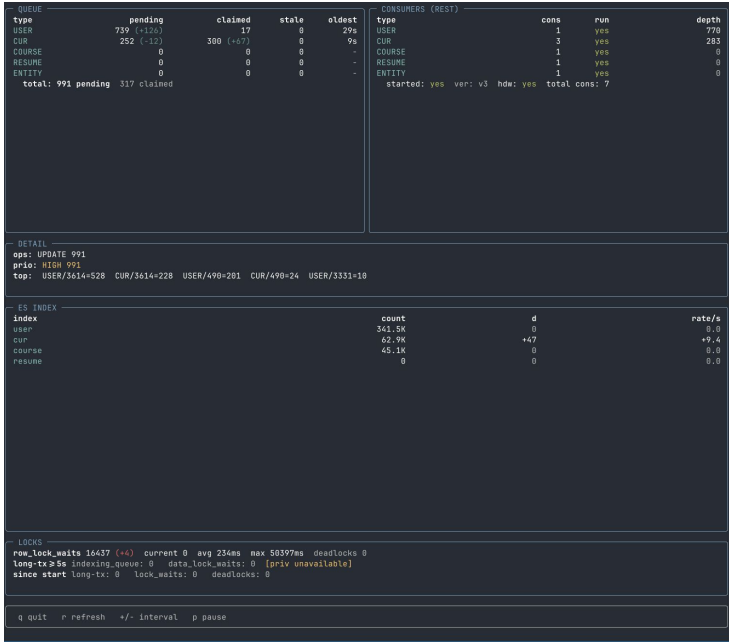
- The production traffic was much higher than anything we tested
- We rolled forward with fixes
- Eventually we needed to roll back
- I didn't keep the old indexer behind a feature flag
- A lovely 10 hours after work, preparing the revert to V1
- We stabilized the system and went to sleep





Critical issue

- V2 was not completely wrong
- But production showed us what our tests didn’t
- I tried to move V2 forward
- I generated a TUI app to see:
  - Elasticsearch
  - Locking
  - Performance of indexing
- But **I still couldn’t reproduce the production issue** properly



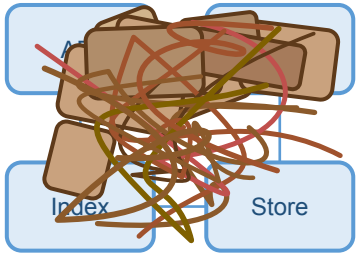
The magical “break system” function

- Someone showed me a maintenance feature that floods the JMS and indexing queues
- Finally, I could reproduce the issue locally, at least partially
- But every attempt to avoid locks/slowness in V2 failed
- At some point I was done with V2 …



What was wrong with V2?

- I generated way too much code
- It became too complicated to deal with
- There were too many heavy tests
- Feedback loops became slower
- I got too many dopamine shots from generating stuff
- Experience told me that the solution should have been MUCH simpler
- I didn’t keep any quick rollback option



What I got

AI slop



Building the harness

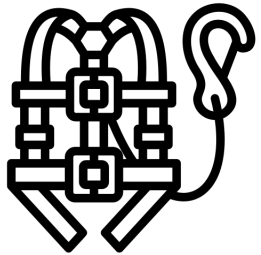
AI needs a harness

- **Traceability**: small sessions, plan and progress files
- **Verification**: tests plus production-like load and profiling, definition of done
- **Review**: other models challenge it, a person verifies
- **Limited access**: agents get beta/staging, production stays fully separate
- **Safety net**: feature flag / rollback path, monitoring



The harness, in practice

- Ralph loops with small sessions: prompt.md, plan.md, progress.md
- Fast tests, kept deliberately light this time
- A local "break system" run as the real stress test
- A TUI and dashboard to watch queues and locks
- Me, reviewing every few iterations



The harness is everything around the model: tests, checks and people.

"Tests are green" is not "done"

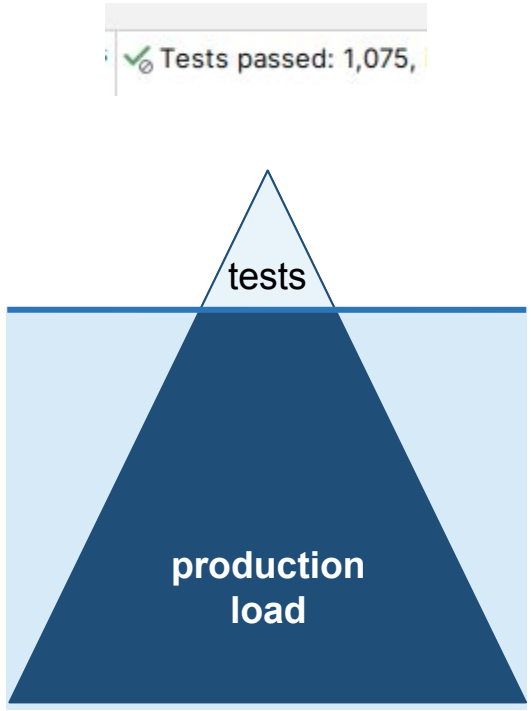
V2: done =

- Many tests green
- Heavy stress testing, but far below production load

V3: done =

- Tests green
- "Break system" run locally at production-like load: no locks, no slowdown
- Manual checks by me afterwards

Done means it survives production-like load, not just my tests.



Is it the right design?



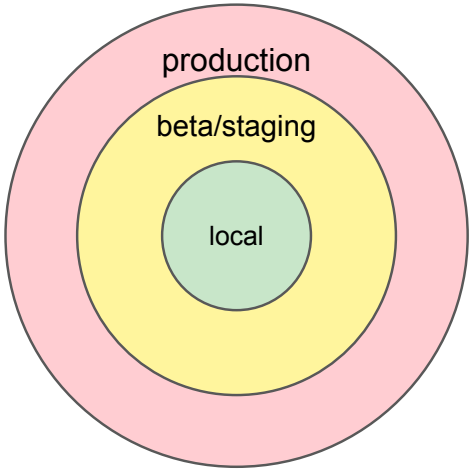
- I asked several models to challenge the plan and the implementation:
  - Does it fulfill the ADR?
  - Does it avoid V2's issues?
  - Where are the transaction boundaries?
- **Result**: many small transactions with short timeouts, instead of long locks

An agent shouldn’t be the only one checking its own work.

Trade-offs … as always

What can the agent actually reach?

- **My V3 agents:**
  - local machine only: Maven, local DB, Testcontainers
  - no VPN
  - no sensitive resources
- **Dev agents:**
  - beta and staging via VPN
  - never production
- **People with prod access:**
  - a separate account
  - Just-In-Time only (still rolling out)



What the agent can log in to matters more than where it runs.

Fewer mistakes vs. smaller mistakes

Fewer mistakes

Nice to have. No guarantee.

- Same agent setup for everyone, part of onboarding (Claude Code, Codex)
- Run agents in a sandbox
- Deny rules for risky commands

Smaller mistakes

This is what actually protects you.

- Agents can’t touch production
- Feature flags and a quick rollback
- Small MRs that someone really reads

Settings help. What really protects you: no production access and a quick way back.

The safety net

	V2	V3
Feature flag	None, all or nothing	On/off per index type, Start/Stop from the dashboard
Rollback	Redeploy the previous version	Switch the flag, old path still there
Observability	General monitoring, blind to locks and queue backlog	Indexing dashboard: queue depth, throughput, success rate, failed rows, staleness alerts



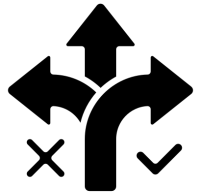
Assume it will fail, so make failing cheap.

V3: simpler, safer, in production

Three alternatives

“Generate at least three alternatives.” - Eric Evans

- AI makes multiple designs and POCs cheap
- Compare them side by side
- Delete generated work when it doesn't fit



Options are cheap to generate now. Don’t fall in love with the first one.

V3 implementation

- I updated the ADR. V3 MUST BE:
  - Quite SIMPLE
  - As close to no-lock implementation as possible
  - Behind a feature flag
  - With old V1 indexer still available for quick rollback
- I created several plans with all models I had access to
- Cross-checked them with myself and against each other
- One plan caught my eye: claim-check pattern, many small steps, simpler overall



V3 was simpler, not simple



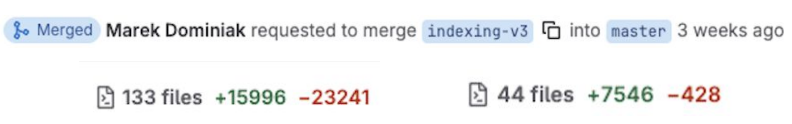
- **Trade-offs:**
  - Approximate fairness
  - Allow harmless duplicates
  - Recovery/retry instead of clever locking

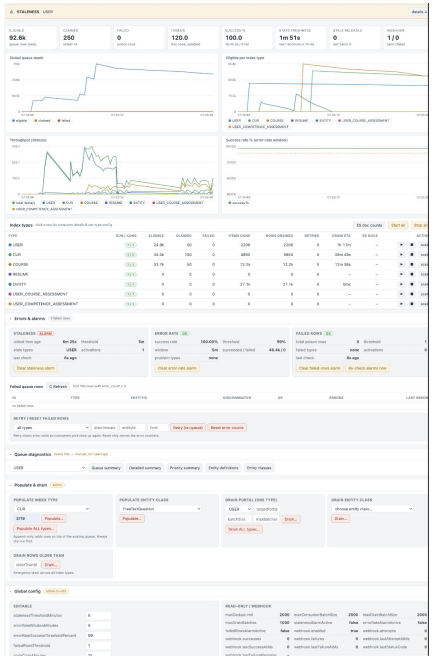


AI helped implement, and the team accepted the trade-offs.

Bulletproofing V3

- Each risky part of indexing got extra attention
- “Break system” must not reproduce V2 issues
- Tested heavily again
- Showed it to the team
- We put it on beta
- Generated one HTML + JS dashboard for monitoring and control





Release expectation

On 29 July we turn on the flag and we expect



In the worst case, we have a very quick rollback option

Three months have passed ….

V3 in production

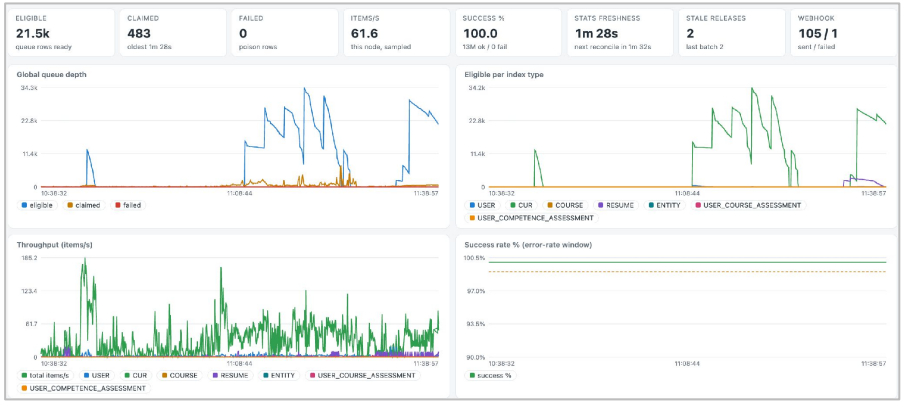
- I expected fireworks, but it was boring!
- Running in production for >3 months
- Runs continuously, often dozens of millions of items a day
- There were some smaller issues, but we have dealt with them
- Full visibility: queues, throughput, errors, staleness
- Feature flag and rollback still available



What production looks like



Draining a backlog of ~800k items



Steady state: 100% success, 0 failed rows

Observability!

From V1 to V3



DB queue, Quartz, JMS, listeners

Too many moving parts

Lost control of priorities

No JMS, priorities, fairness, dedup

Too much code, heavy tests

Released with no way back

Thrown away along the way:
most of the V2 code and its TUI

Same goals, much simpler

Small transactions, claim-check

Behind a flag, V1 kept as fallback

Each version learned from the last. The one real mistake: no way back for V2.

Lessons Learned

Enterprise AI models are not as stable as I would like

- Models change their performance
- Models are deprecated
- Thinking time can change
- Tool wrappers can make models behave differently (Claude Code vs API)
- Access can change anytime: vendors, platforms, regulation (e.g. Fable 5)
- Keep plans, specs and ADRs outside the tool



Don't build your whole process around one magical model / tool.

Treat tokens as a scarce resource

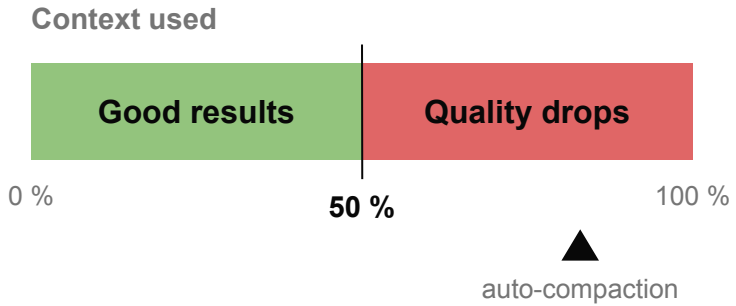
- **Today:** flat-rate subscriptions, it can feel unlimited
- **Make the most of it while it lasts:** experiment and try several designs
- **In the future:** tokens get budgeted like CPU, memory or cloud spend
- **Many companies** already track token cost per team
- **Engineer around it:** small sessions, cheaper models for simple tasks, reusable plans and skills



Plan for tokens the way you plan for CPU and memory.

Keep sessions small (still)

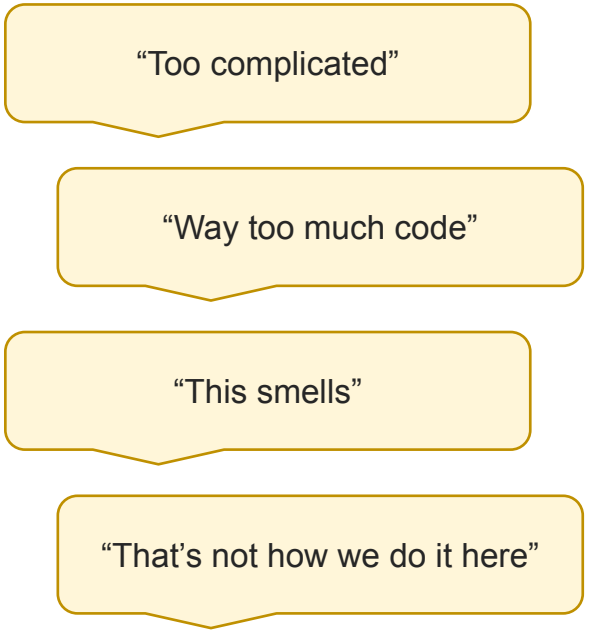
- Ralph loops and small sessions work even with cheaper models
- Opus 5.5 can run for hours on its own
- But auto-compaction drops details: don't trust it blindly
- Check /context. Prefer a fresh session with a plan file



After half the context, quality drops.

Listen to your code reviews

- Early on, a lot of weird code reached our merge requests
- In TDD you listen to your tests. Here we listen to our reviews
- A gut reaction in review is a code smell worth writing down
- Each one becomes a rule or a skill, linked from CLAUDE.md / AGENTS.md
- Our rule now: you must understand and explain every line of your MR



Honest reviews, even when they’re not nice: that’s our verification tax.
We pay it on purpose.

Vibe-coded is fine if someone understands it, owns it, and it survives production.

Teach the agent once: rules and skills

CLAUDE.md / AGENTS.md: read in every session

```
### Comments
- Explain why, not what.
- Keep them short.

## Minimizing Code Growth
- Before creating a new class, look for an
  existing one you can extend or reuse.

## Reusing Existing Integration Tests
- Prefer a new test method in an existing class.
- No new @ContextConfiguration or
@DirtiesContext:
  each forces a second Spring context.
```

Skills: team know-how it can run

/migration-risk

Which Liquibase changesets can lock hot tables, and which can run by hand before the deploy. Used for production releases since March.

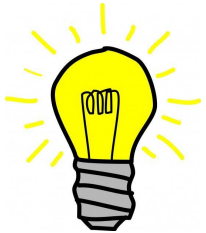
/idempotent-db-change

Writes an index or data change guarded by preConditions (MARK_RAN), then proves it on a scratch schema.

Rules and skills live in the repo: peer-reviewed like code.

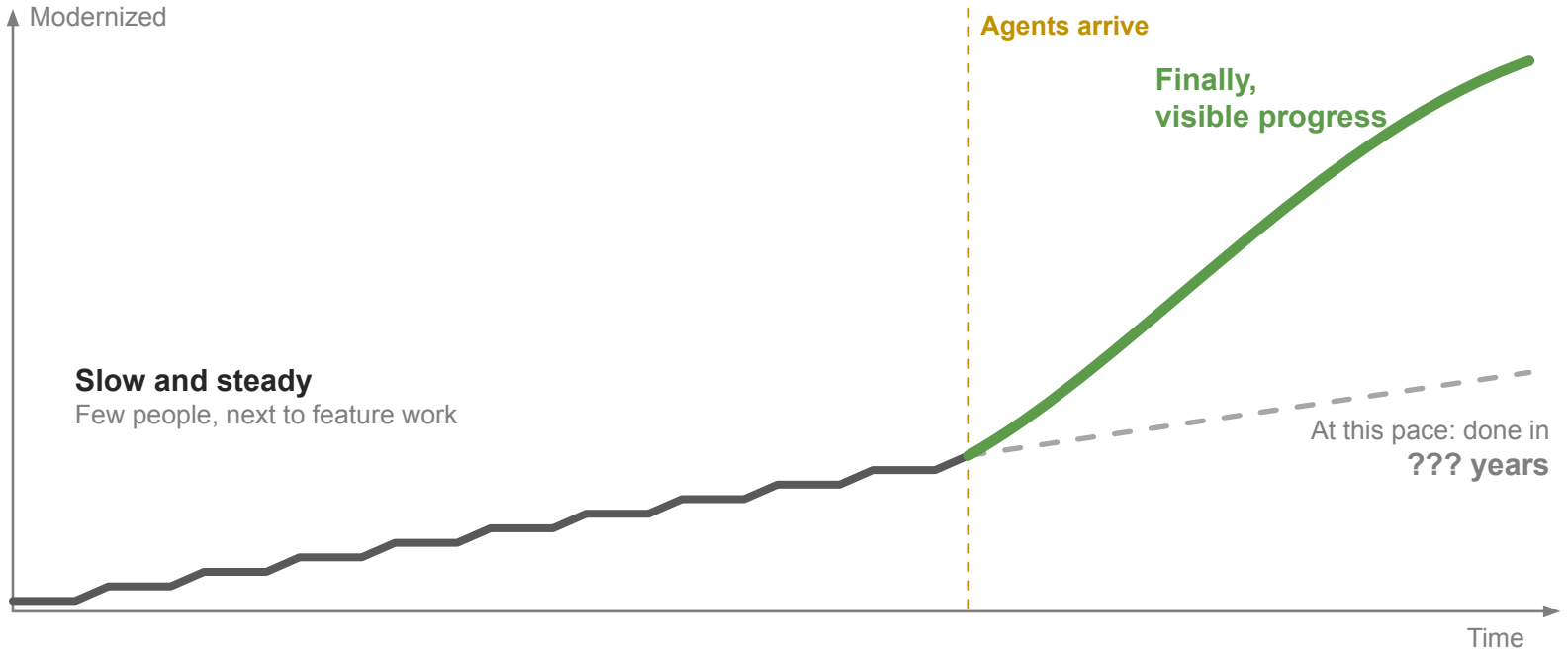
Agentic AI is a double-edged sword

- It can help you remove tech debt and make progress
- Or you can generate AI slop and create even more tech debt
- AI is a multiplier of your capabilities
    - Wrong direction → you move away from the target 100x faster
    - Right direction → you move closer 100x faster



Keep a person in the loop every few iterations.

The light at the end of the tunnel



Illustrative, not measured.

For years we couldn’t say when we’d be done. Now we can see it.

You get paid to think.

The agent writes the code. You decide what it should do, challenge it, and own the result.

Don’t be a meat proxy.

Q/A

trainitek



marekdominiak

Presentation Slides

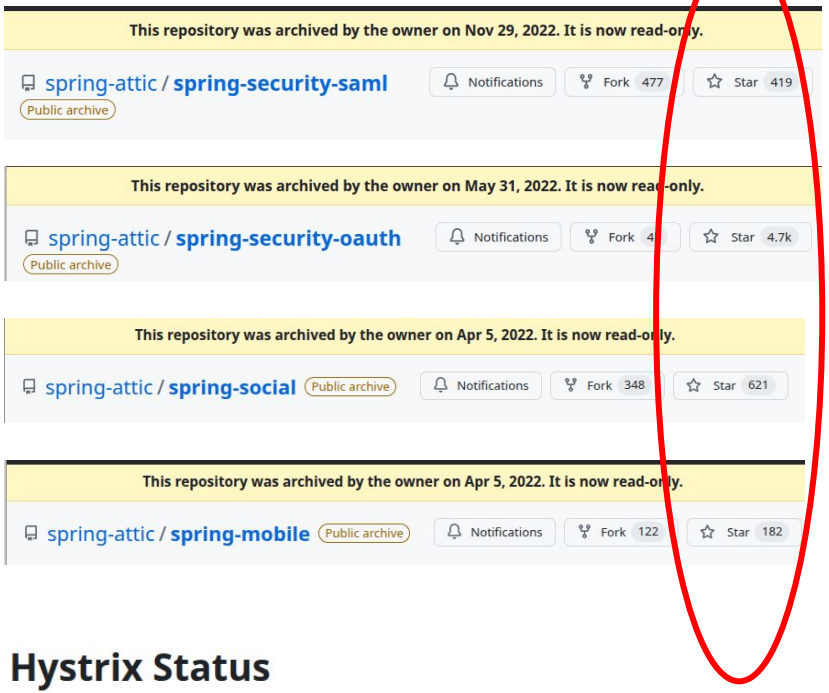
“Easy” upgrades

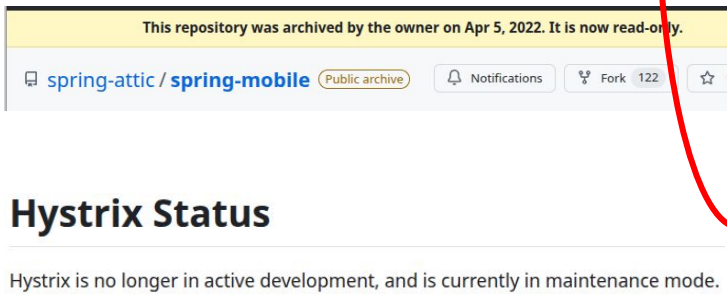
- Java 11 to Java 21
- Tomcat 9 to Tomcat 11
- Spring Framework (Core, MVC) 4.3.x to 6.x



Libraries that disappeared

- Spring-security-saml
- Spring-security-oauth
- Spring-social
- Spring-mobile
- Springockito
- Hystrix
- Fongo
- Spring-Velocity





What V2 addressed

- **Fewer moving parts**: no JMS, the queue is just database tables
- **Priorities and fairness**: urgent changes first, and one big customer can't block the small ones
- **Duplicates**: the same entity queued many times gets indexed once
- **Much better monitoring**
- **Operator tools**: REST endpoints and Postman collections to start, stop and scale indexers without restarts



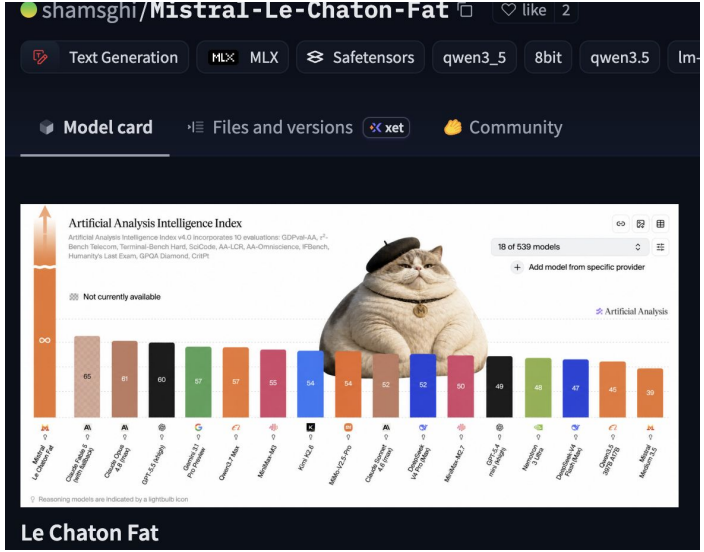
This time I restrained myself

- I didn’t let the implementation grow
- I didn’t let the tests become slow
- I accepted and documented the trade-offs
- I documented potential optimizations for the future
- My mindset was wary of potential issues



Don’t vendor lock yourself

- Be ready to switch models/tools
- Keep workflow portable
- Use plans/specs/ADRs outside the tool
- Keep session sizes small
- IDE (IntelliJ) less and less: mostly for review, analysis, and checks before push
- I still hope for strong European-hosted models



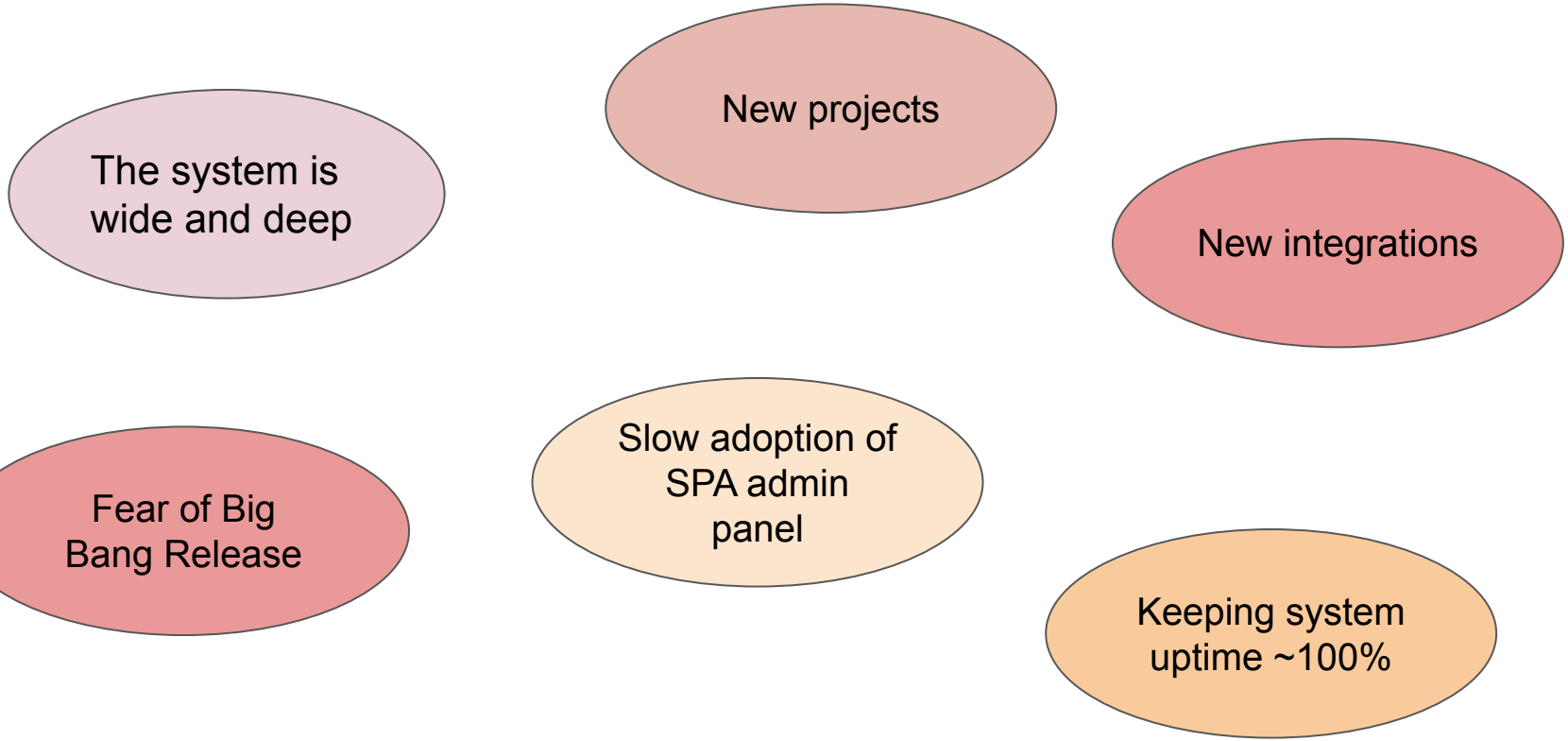
Your process should outlive your favorite model.

Please rate my talk

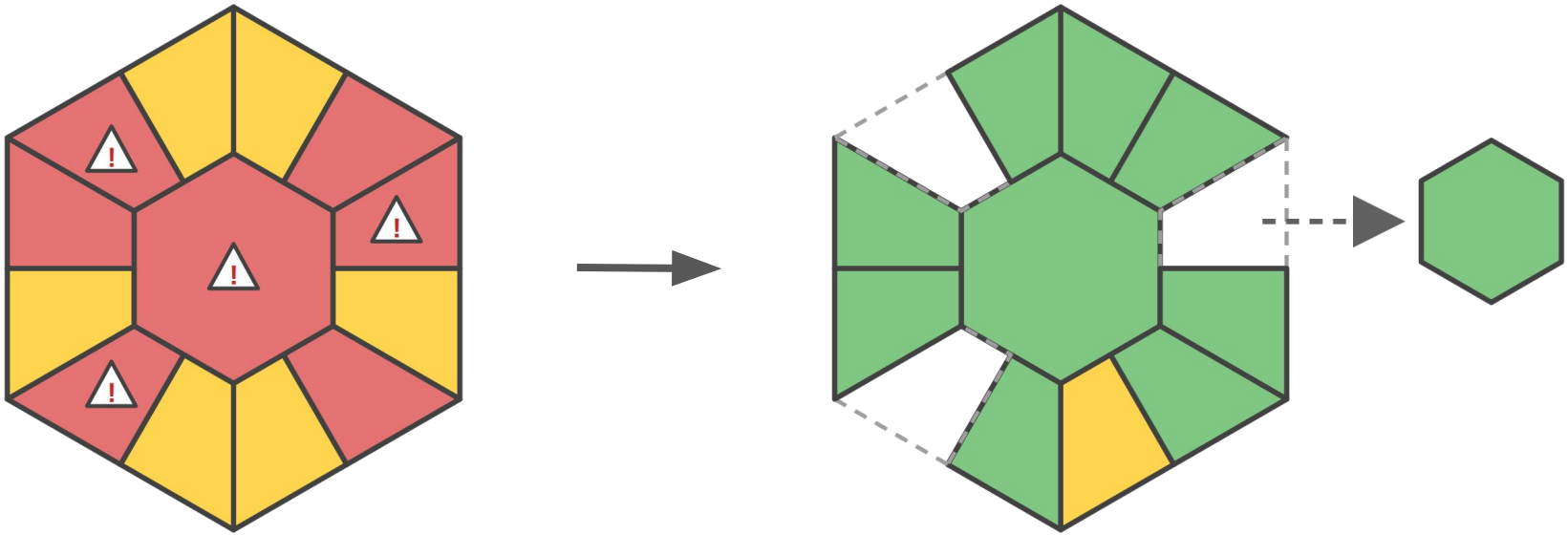


Finally, Agentic AI Helped Us Modernize Our 2M+ LOC Monolith

What held us back?

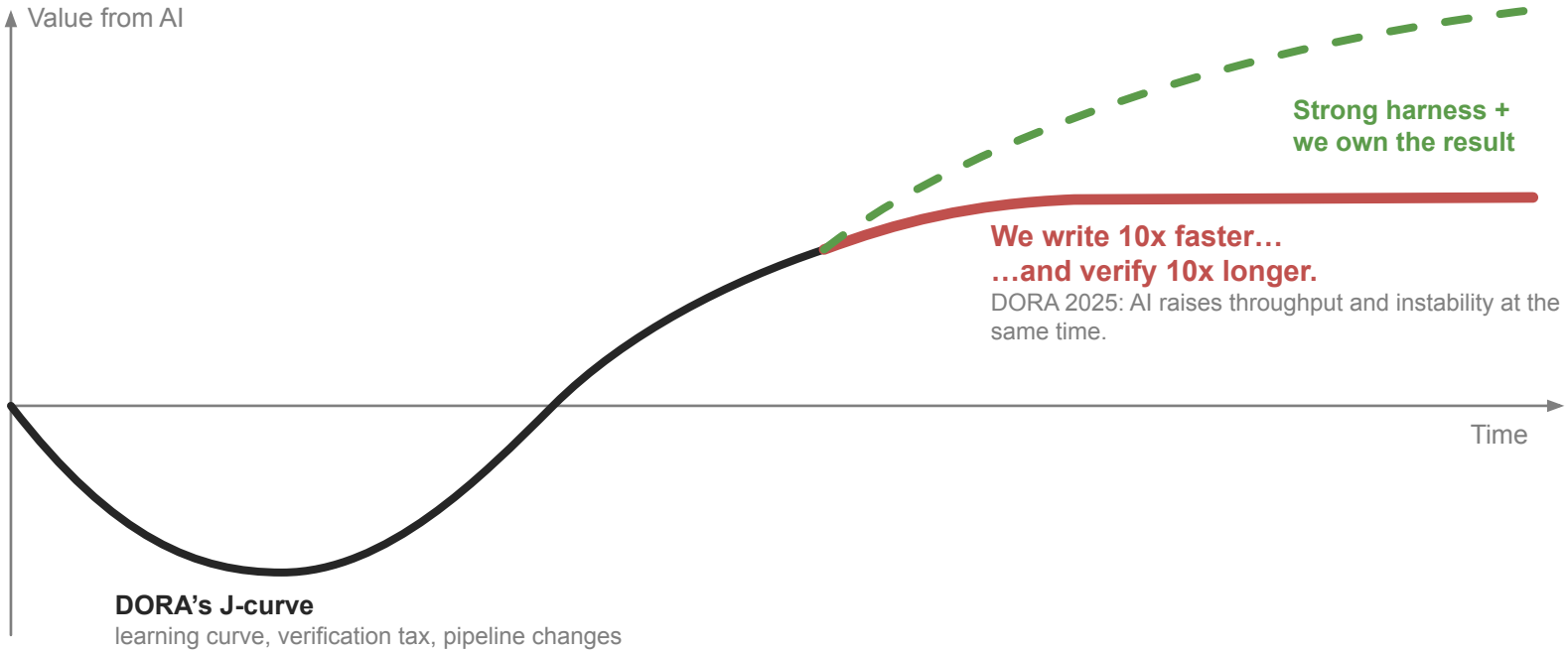


The light at the end of the tunnel



We started healing it: not one big effort, but many small ones.

The verification tax



Sources: DORA, State of AI-assisted Software Development (2025); ROI of AI-Assisted Software Development (2026).

Code got cheap to write. Checking it and owning it still takes time.